

Delta Design & Implementation Report

BIL482 – Software Design | Fantasy Premier League Team Builder | April 2026

1. Introduction

During the review sessions, feedback was collected on architectural decisions, design-pattern choices, and overall code quality. Given the remaining implementation window of roughly 4–5 days and a realistic daily commitment of 1.5–2.5 hours alongside other course workloads, one improvement was identified as both achievable and genuinely impactful: making the Singleton pattern thread-safe.

2. Selected Improvement

Feedback received: "Thread safe yapabilirsiniz singleton'ı, o kolaydır."
(*Make the Singleton thread-safe — it is easy.*)

2.1 Justification

The project already used a Singleton for both **DatabaseManager** and **APIService**. Adding thread-safety did not require a new abstraction layer or significant architectural restructuring — it was a targeted, self-contained change with a clear, verifiable outcome.

The problem being solved is real: in any environment where multiple requests arrive concurrently (e.g., the Flask API handling several users building teams at the same time), the naive Singleton can be instantiated more than once. This leads to redundant database connections or conflicting data-fetch operations — exactly the kind of issue Singleton is supposed to prevent.

2.2 Effort Estimation & Cost Calculation

Task	Estimated Effort
Implementation & code refinement	~0.5 hours
Testing & bug fixing	~1.0 hour
Documentation	~0.5 hours

Task	Estimated Effort
Total	~2.0 hours

With 1.5–2.5 hours available per day over 4–5 days the total budget is **6–12.5 person-hours**. The ~2-hour cost of this improvement consumes at most one-third of a single day's allocation, leaving the remaining days free for testing, bug-fixing, and report finalisation.

3. Implementation Details

3.1 Architectural Update

The change is confined to `code/fantasy_team_usecase_1/singletons.py`. The previous implementation used a bare instance check with no synchronisation primitive. The new version introduces a **SingletonMeta** metaclass that implements **Double-Checked Locking** via Python's `threading.Lock`.

3.2 Design Decision — Why Double-Checked Locking?

A straightforward with `cls._lock`: guard around the entire block would be correct, but it forces every subsequent call to acquire the lock — even after the instance already exists. Under high concurrency this becomes an unnecessary serialisation bottleneck. Double-Checked Locking avoids this: the lock is acquired only when the instance has not yet been created. Once initialised, all threads read the existing instance without any synchronisation overhead.

```
class SingletonMeta(type):
    _instances = {}
    _lock = threading.Lock()

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            with cls._lock:
                if cls not in cls._instances:
                    instance = super().__call__(*args, **kwargs)
                    cls._instances[cls] = instance
        return cls._instances[cls]
```

Both **DatabaseManager** and **APIService** continue to use `metaclass=SingletonMeta` unchanged. Thread-safety is therefore inherited automatically by any future class that adopts the same metaclass.

3.3 Impact on Quality Metrics

Quality Attribute	Before	After
Reliability	Race condition possible during concurrent startup	Only one instance can ever be created
Performance	No lock overhead (but unsafe)	Lock acquired on first init only; subsequent calls are lock-free

Quality Attribute	Before	After
Design Pattern	Basic Singleton	Thread-Safe Singleton via Double-Checked Locking
Testability	Race condition hard to reproduce deterministically	50-thread concurrency test provides a reproducible baseline

4. Test Report & Bug-Fixing

The last 1–2 days of the timeline were dedicated to validating the new implementation and resolving issues discovered in the process. A dedicated test file, `tests/test_singleton_concurrency.py`, was written for this purpose.

4.1 Test Scenario

50 threads are spawned simultaneously, each calling `DatabaseManager()`. Every thread records `id()` of the instance it receives. After all threads finish, the set of unique IDs is inspected: cardinality of 1 means the Singleton is thread-safe.

```
def worker(results_list, index):
    results_list[index] = id(DatabaseManager())

results = [None] * 50
threads = [threading.Thread(target=worker, args=(results, i)) for i in range(50)]
for t in threads: t.start()
for t in threads: t.join()

assert len(set(results)) == 1, 'Multiple instances detected!'
```

4.2 Test Output

```
Total threads run      : 50
Unique instance IDs    : 1
Result                 : SUCCESS -- All 50 threads received the identical instance.
```

4.3 Bug / Fix Report

Bug ID	Description	Severity	Fix Applied	Status
BUG-001	Race condition in <code>DatabaseManager</code> instantiation: concurrent calls could produce multiple DB connections before any instance was cached.	High	Implemented <code>threading.Lock</code> via <code>SingletonMeta</code> with Double-Checked Locking.	FIXED
BUG-002	<code>test_singleton_concurrency.py</code> failing with <code>ModuleNotFoundError</code> — project root not on <code>sys.path</code> during test execution.	Low	Added <code>sys.path.insert(0, ...)</code> at the top of the test file.	FIXED

Conclusion: The concurrency test passed without a single failure across all 50 threads. Every thread received the same instance object, confirming that the Double-Checked Locking implementation is correct under concurrent access. No regressions were observed in the existing use-case tests.